- PROJECT SNAPSHOT AND FEATURES

- LexiRisk AI: academic prototype for AI-assisted legal contract analysis and preliminary risk screening.
- Primary goal: turn uploaded contract text into clause-level findings, evidence, risk scores, and review actions.
- Supported domains: general legal/commercial contracts, real estate, and construction.
- Document screening: accept, accept with warning, or reject likely out-of-domain documents.
- Ingestion: PDF, DOCX, and TXT are supported by the general upload endpoint; direct analysis upload accepts PDF and TXT.
- Clause workflow: segment contract text, assign one of 15 clause categories, and retain page and clause metadata.
- Risk workflow: rule checks, risk phrase matching, missing-clause checks, aggregate scoring, explanations, and recommendations.
- Persistence and exports: save analyzed contracts, clauses, and risks in SQLite; retrieve HTML and JSON reports.
- Demo options: bundled single-page web UI, Streamlit dashboard, and four built-in sample documents.
- Safety position: screening aid only; not legal advice and not a replacement for qualified counsel.

- TECH STACK AND RUNTIME

- Language: Python; current workspace interpreter is Python 3.13.4.
- API framework: FastAPI with Pydantic request models and Uvicorn as the ASGI server.
- Web interface: HTML, CSS, and JavaScript served as static assets by FastAPI.
- Demo interface: Streamlit with Requests for calls to the local API.
- Document parsing: pypdf for PDF text extraction; python-docx for DOCX paragraph extraction.
- Text and classification: Python regular expressions, phrase/heading signatures, and scikit-learn TF-IDF.
- Numerical operations: NumPy; persistent storage: SQLite through Python’s sqlite3 module.
- Declared requirements also include Transformers, Torch, PyMuPDF, pandas, and joblib.
  - The reviewed analysis path does not load BART, sentence-transformers, a trained classifier, or a generative LLM.
- PDF text extraction is not OCR; image-only/scanned PDFs may produce no usable text.
- Upload limit in API handlers: 25 MB.

- END-TO-END SYSTEM DESIGN

- 1. User selects a PDF, DOCX, or TXT document, or chooses a sample contract.
- 2. FastAPI validates the upload and extracts page text with page numbers and paragraph data.
- 3. Preprocessing normalizes whitespace and removes simple page-number noise while preserving structure.
- 4. Domain screening scores keyword signals and rejects clear non-legal/out-of-domain content.
- 5. Clause segmentation detects headings, numbering, and paragraph boundaries.
- 6. Clause classification applies heading, keyword, and regular-expression signatures.
- 7. Risk modules identify risky terms and missing expected clauses, retaining evidence and page context.
- 8. Scoring combines rule, category-density, heuristic assessment, and missing-clause components.
- 9. Explanation templates produce plain-language summaries and suggested review actions.
- 10. Contract, clause, and risk records are saved; the API returns JSON for the UI and report views.
- Design pattern: layered modular monolith; routers coordinate focused Python modules in one FastAPI process.
- Primary data path: UI -> API router -> ingestion/preprocessing -> screening/segmentation/classification -> risk/scoring -> SQLite -> JSON/report.

- API SURFACE AND DATA MODEL

- POST /api/upload: validate and parse PDF, DOCX, or TXT; return cleaned text, page data, and readability.
- POST /api/analyze: run the standard pipeline and return screening, clauses, risks, score, and summary.
- POST /api/analyze/detailed: add keyword risk scores, matched factors, and a risk summary.
- POST /api/analyze/upload: accept PDF or TXT directly and run detailed analysis.
- POST /api/screen-domain: run domain screening without full analysis.
- GET /api/samples: return demo samples; GET /api/contracts and /api/contracts/{id}: browse saved analyses.
- DELETE /api/contracts/{id}: delete a saved contract and related records.
- GET /api/reports/{id}/html and /json: retrieve printable HTML or structured JSON reports.
- SQLite location: backend/app/data/contracts.db.
- contracts table: file/type/domain/status/date, score, risk level, confidence, summary, metadata, and text excerpt.
- clauses table: contract link, clause number/type/text, page, confidence, and character offsets.
- risks table: contract/clause link, risk type/level, reason, evidence, recommendation, and score weight.

- RISK AND EXPLAINABILITY METHODS

- 15 classifier labels: Parties, Effective Date, Expiration Date, Termination, Notice, Renewal, Payment, Penalties, Indemnification, Limitation of Liability, Governing Law, Dispute Resolution, Assignment, Obligations, and Confidentiality.
- Clause labels come from rule-based signatures; confidence is a heuristic score, not a calibrated model probability.
- Detailed risk patterns cover Termination, Indemnification, Payment, Confidentiality, Liability, Force Majeure, Governing Law, Dispute Resolution, Penalties, and Renewal.
- Phrase tiers produce LOW, MEDIUM, or HIGH clause risk and record matched risk factors.
- Detailed missing-clause checks: Force Majeure and Limitation of Liability (HIGH); Dispute Resolution and Confidentiality (MEDIUM).
- Detailed overall risk: average clause risk plus missing-clause penalties, clamped to 0.0-1.0.
- Standard hybrid score: 0.40 rule score + 0.30 category-density score + 0.20 heuristic assessment + 0.10 missing-clause score.
    - Standard score bands: Low below 31; Medium from 31 to below 61; High at 61 and above, on a 0-100 scale.
- Evidence fields connect findings to clause text, page, triggered rule, reason, and recommended review.
- The LLM analyzer module currently uses deterministic templates; the score component labeled LLM is heuristic, not model inference.
- The embedding module uses TF-IDF vectors and cosine similarity; no transformer embeddings or K-Means pipeline is wired into analysis.

- CODEBASE AND FILE STRUCTURE

- mvpp/README.md: project overview, current architecture, setup, sample walkthrough, and database schema.
- mvpp/legal_contract_risk_analyzer_mvp_prompt.md: feature requirements for a related/upgraded MVP.
- mvpp/kilo_master_prompt.md: separate proposed transformer/LEDGAR ML design; treat as planned, not current behavior.
- mvpp/streamlit_app.py: presentation demo UI that calls the FastAPI API.
- backend/app/main.py: FastAPI app, startup database initialization, router registration, and static UI hosting.
- backend/app/api/: upload.py, analysis.py, contracts.py, and report.py define the HTTP endpoints.
- backend/app/ingestion/: pdf_parser.py, docx_parser.py, and ocr.py handle extraction/readability checks.
- backend/app/preprocessing/: cleaner.py and normalizer.py prepare and normalize contract text.
- backend/app/screening/ and segmentation/: domain_classifier.py and clause_segmenter.py.
- backend/app/models/: clause_classifier.py, embeddings.py, and contract.py.
- backend/app/risk/: rules.py, detector.py, missing_clauses.py, and scorer.py.
- backend/app/llm/: prompts.py and analyzer.py provide disclaimers and deterministic explanations.
- backend/app/database.py and app/crud/: SQLite persistence and contract CRUD helpers.
- backend/app/reports/generator.py: HTML report creation; backend/app/static/: index.html, css/style.css, and js/app.js.
- backend/app/sample_contracts.py and backend/tests/test_pipeline.py: example contracts and integration-style checks.

- SPEC STATUS AND PRESENTATION NOTES

- Implemented and visible in code: three-domain screening, 15 signature-based clause labels, risk rules, missing-clause checks, SQLite history, web UI, Streamlit demo, and HTML/JSON reports.
- The README and current code include persistence and report exports; an older prompt lists these as exclusions for a different MVP scope.
- Older prompts mention BART zero-shot classification, sentence-transformer embeddings, LEDGAR training, K-Means, six domains, and a Next.js frontend.
  - Those older prompt items are not the active implementation inspected in backend/app; avoid presenting them as working features.
- The college proposal DOCX could not be parsed as a valid DOCX archive; its contents were not used as verified requirements.
- Demo examples: lease agreement, NDA, construction subcontract, and an academic paper used as a rejection example.
- Recommended demo sequence: upload/select sample -> show screening -> inspect clause categories -> open evidence/risk explanation -> show aggregate score -> browse saved contract/report.
- Discuss limitations: heuristic classification/scoring, no OCR for scanned documents, no external legal-data validation, and no legal advice.
- Run backend from mvpp root: python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --reload.
- Open http://localhost:8000 for the bundled UI and http://localhost:8000/docs for interactive API documentation.
- Run the demo UI from mvpp root: streamlit run streamlit_app.py; keep the FastAPI server running.